

2025年度
修士論文

SN開発におけるデバッグシステムの開発

指導教員	アサノ デービッド	教授
	不破 泰	肩書き

2025年1月30日提出

信州大学大学院 総合理工学研究科 工学専攻 情報数理・融合システム分野
24W6047A
辻村篤志

あらまし

あらましを書く。

目次

第1章	はじめに	1
1.1	研究背景	1
1.2	当研究室における先行研究	2
1.3	先行研究の課題	2
1.4	本研究の目的	2
1.5	本論文の構成	2
第2章	先行研究の提案システム	4
2.1	システムの概要	4
2.2	実現事項	5
2.3	課題	5
第3章	提案システムの概要と設計方針	6
3.1	設計方針	6
3.2	システムの構成	7
3.3	各コンポーネントの概要	7
3.3.1	コード生成コンポーネント	8
3.3.2	組込みシステムコンポーネント	8
3.3.3	デバッグ支援システムコンポーネント	9
第4章	デバッグ支援システムの設計・実装	10
4.1	デバッグ支援システムの概要	10
4.2	実装技術と採用理由	10
4.2.1	フロントエンドフレームワーク	10
4.2.2	UI フレームワーク	11

4.3	動作モードとシステム構成	11
4.3.1	動作モード	11
4.3.2	モード間で共通に用いる要素	11
4.4	ログモードの設計・実装	12
4.4.1	ログモードの概要	12
4.4.2	ログモードの UI 設計	12
4.5	同期モードの設計・実装	13
4.5.1	同期モードの概要	13
4.5.2	同期モードの UI 設計	13
4.5.3	ブラウザ上でのシリアルモニタ実装	15
4.5.4	同期タイムラインの生成	16
4.6	ログ設計	17
4.6.1	ログ生成の設計	17
4.7	ログ解析の設計（共通処理）	18
4.7.1	概要（解析の目的と処理の流れ）	18
4.7.2	入力ログ形式とログ行の分類	19
4.7.3	解析結果として構築するデータ構造	20
4.7.4	変数値スナップショットの再構成方針（継承と更新）	20
4.7.5	行処理アルゴリズム（擬似コード）	21
4.7.6	解析結果例	22
4.8	ステップ番号に基づく共通表示ロジック	22
4.8.1	共通の前提：ステップ番号 i と状態スナップショット	22
4.8.2	ステップ実行のロジック	23
4.8.3	変数ウォッチ機能のロジック	24
4.8.4	状態遷移表示機能（ハイライト表示）	24
4.8.5	Mermaid による状態遷移図描画（構文生成ロジック）	25
第5章	評価	27
5.1	評価の目的と方針	27
5.1.1	評価の目的（何を確かめたいか）	27
5.1.2	評価対象	27

5.1.3	評価で用いるデータ（実験対象）	28
5.2	単一ノードデバッグの評価（ログモード／単一ノード受信の共通部分）	28
5.2.1	評価シナリオ	28
5.2.2	状態遷移の可視化（ハイライトとログ表示）	28
5.2.3	変数ウォッチ機能（状態ステップに対応した変数値追跡）	29
5.2.4	ステップ実行（逐次確認）と任意ステップへの移動	29
5.3	同期モード固有機能の評価（2ノード同期タイムライン）	29
5.3.1	評価シナリオ	29
5.3.2	同期タイムライン表示	29
5.3.3	複数ノード比較の有効性（定性的観察）	29
5.4	定性的評価	30
5.4.1	単一ノードデバッグに対する効果	30
5.4.2	同期デバッグに対する効果	30
5.4.3	まとめ	30
第6章	考察	31
6.1	有効性の要因	31
6.1.1	状態遷移単位への再構成	31
6.1.2	変数値スナップショットの継承と更新	31
6.1.3	同期モードにおける統合表示	31
6.2	システムの限界・制約	32
6.2.1	ログ設計への依存	32
6.2.2	同期タイムラインに関する制約	32
6.3	今後の課題	32
6.3.1	ログ形式・解析対象の拡張	32
6.3.2	同期精度および比較支援の拡張	32
6.3.3	状態遷移図表現への対応	32
第7章	まとめ	33
7.1	本研究のまとめ	33
7.2	得られた知見と今後の展望	33

図 目 次

2.1	先行研究システムの概要図	4
3.1	従来のデバッグ方法	7
3.2	提案システムの概要図	8
3.3	どこでモデム (左) と GPS モジュール (右)	9
4.1	ログモードのレイアウト	13
4.2	シリアルモニタのレイアウト	14
4.3	同期モードのレイアウト	15
4.4	同期モードのタイムライン	16
4.5	同期モードのタイムライン	16
4.6	ログ解析の処理フロー	19
4.7	変数値スナップショットの継承と更新	21

表 目 次

第 1 章

はじめに

はじめにを書く。

1.1 研究背景

近年、無線センサネットワーク（Wireless Sensor Network：WSN）や IoT（Internet of Things）は、環境モニタリングやスマートシティ、インフラ監視、農業、ヘルスケアなど、さまざまな分野での応用が期待されている。実際、IoT 全体の接続デバイス数は急速に増加している。IoT Analytics によれば、2023 年時点で約 166 億台、2024 年には約 188 億台に達し、2030 年には約 400 億台に到達すると予測されている [1]。また、WSN の普及も市場規模の面から拡大を続けており、Grand View Research の報告では、産業用 WSN（Industrial WSN）の市場は 2023 年に約 51.9 億ドルと推定され、2024 年から 2030 年にかけて年平均 12.1%の成長が見込まれている [2]。

しかし、これらのシステムを構築するためには、複雑な通信プロトコルの設計やデータ処理の設計および実装が求められ、開発者には幅広い専門知識と多大な工数が必要となる。さらに、無線通信環境の構築やマイコン設定など導入時のハードルも高く、これらの研究や開発の多くはシミュレーション環境やエミュレータ上での検証にとどまることが多い。しかし、シミュレーション環境では実機特有の挙動や外部環境の影響を完全に再現することが難しく、実機での検証が不可欠である場合も多く、実機でのプロトコル検証を容易にするための支援環境の整備が求められている。

1.2 当研究室における先行研究

当研究室の先行研究では、状態遷移図上で無線通信プロトコルの動作を定義し、そこから自動生成したプログラムを汎用ハード上で動作させ、プロトコルを実機で動作検証できる環境が構築されていた（旭ら [3]、小林ら [4]）。

1.3 先行研究の課題

このシステムにより基本的な通信プロトコルの動作検証が可能となったが、デバッグ方法はコンソールへのログ出力に依存しており、通信処理が高速に進むため逐次的な状態遷移を追跡することが難しかった。その結果、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらく、開発効率や教育的利用の観点では十分でない点が課題として残されていた。

1.4 本研究の目的

本研究の目的は、先行研究で課題となっていたデバッグ効率の低さを改善し、実機でのプロトコル検証をより効果的に行える環境を実現することである。そのために、無線通信デバイス上で実行された動作をログとして出力し、それを可視化・ステップ実行できる仕組みを開発した。これにより、プロトコルの動作を逐次的に把握でき、異常動作の原因究明を容易にするとともに、教育やチーム開発、応用実証に適した支援環境を提供することを目指す。

1.5 本論文の構成

本論文は全7章から構成されている。第1章では、本研究の背景として当研究室におけるこれまでの研究の流れを整理し、先行研究で顕在化した課題を明らかにした上で、本研究の目的を述べる。第2章では、当研究室における先行研究システムについて、その構成や実現されてきた機能を整理するとともに、運用およびデバッグの観点から課題をまとめる。第3章では、先行研究の課題を踏まえ、本研究で提案するシステムの概要と設計方針について述べる。ここでは、全体構成および各コンポーネントの役割を中心に説

明する。第4章では、提案システムの設計および実装について述べ、特にデバッグ支援を目的としたログ取得方式や状態遷移の可視化手法について詳述する。第5章では、提案システムの操作例を示し、実際の利用を通してどのような情報が得られるかを説明する。第6章では、従来手法との比較を通じて提案システムの有効性を評価し、その結果について考察を行う。最後に第7章では、本研究のまとめを行うとともに、今後の課題および展望について述べる。

第 2 章

先行研究の提案システム

2.1 システムの概要

先行研究では、無線通信プロトコルの設計・実装・動作検証を支援するシステムが提案されてきた。このシステムは主に次の二つのコンポーネントから構成されている。第一に、UML の状態遷移図を用いてプロトコルの動作を定義し、そこからコードを自動生成するコンポーネントである。第二に、生成されたコードを実行するための環境（ハードウェア）を提供するコンポーネントである。これにより、状態遷移図で定義されたプロトコルが実機上で動作し、その動作を検証できる環境が提供されてきた。システムの概要を図 2.1 に示す。

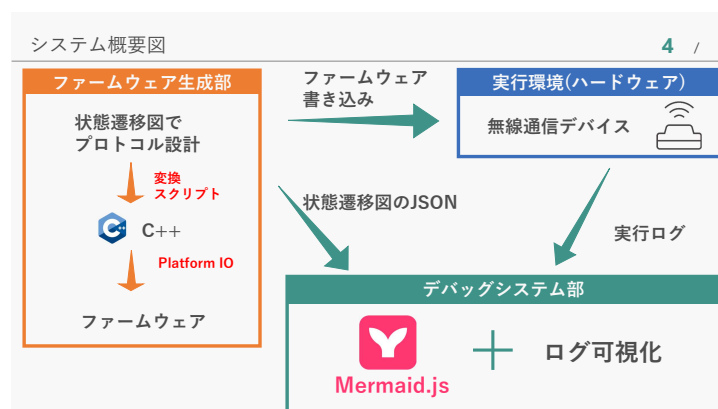


図 2.1: 先行研究システムの概要図

2.2 実現事項

先行研究システムでは、以下の主要な機能が実現されている。

- 状態遷移図からのコード自動生成：UML の状態遷移図を用いてプロトコルの動作を定義し、そこから C 言語コードを自動生成する機能が提供されている。これにより、プロトコル設計者は視覚的に動作を定義でき、手動でのコーディング作業を削減できる。
- 実機上での動作検証：生成されたコードは、無線モデム搭載のマイコンを組み合わせたハードウェア上で実行される。これにより、シミュレーション環境では捉えきれない実機特有の挙動を検証できる。

2.3 課題

先行研究システムには以下の課題が存在する。

- 状態遷移図の記法の制約：先行研究における状態遷移図からのコード自動生成は、特定の記法に依存しており、複雑なプロトコル動作を表現する際に制約がある。これにより、設計者が意図する動作を完全に反映できない場合がある。
- デバッグ効率の低さ：デバッグ方法がコンソールへのログ出力に依存しており、通信処理が高速に進むため逐次的な状態遷移を追跡することが難しい。これにより、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらい。

これらの課題を解決するために、本研究ではデバッグ効率の向上に焦点を当て、実機でのプロトコル検証をより効果的に行える環境を提案する。なお、状態遷移図からのコード生成部については、[?] で改善が図られているため、そちらを参照されたい（たびたびそちらの論文を参照するように指示することがある）。

第 3 章

提案システムの概要と設計方針

本研究では、先行研究システムにおいて課題となっていたデバッグ効率の低さを改善することを目的として、新たなデバッグ支援システムを提案する。本章では、提案システムの概要と設計方針について述べる。はじめに、本研究で対象とする課題を整理し、次に設計方針およびシステム全体の構成について説明する。

3.1 設計方針

先行研究システムでは、状態遷移図を用いて通信プロトコルを設計し、実機上で動作検証を行うことが可能であった。一方で、デバッグ方法は主に図 3.1 のように、コンソールへのログ出力に依存しており、状態遷移の流れや内部動作を逐次的に把握することが難しいという課題があった。

本研究では、この課題を踏まえ、実機上で実行される通信プロトコルの内部動作をより直感的に把握できるデバッグ支援環境の構築を設計方針とした。具体的には、プロトコルの動作を状態遷移として捉え、その実行状況をログとして取得・可視化することで、動作の流れを段階的に確認できる仕組みを目指す。

また、本研究のシステムは、先行研究で構築されたプロトコル設計および実機検証の枠組みを活用することを前提とし、既存システムとの親和性を保ちながら改善を行うことを重視した。これにより、研究室内での継続的な利用や拡張が容易な構成とすることを設計上の方針とした。

```
STATE:IDLE
current_slot = 0
STATE:RECEIVE_SLOT_INFO
STATE:CHECK_RECEIVED_PACKET
STATE:WAIT_OWN_SLOT
current_slot = 1
STATE:WAIT_OWN_SLOT
STATE:WAIT_OWN_SLOT
current_slot = 2
STATE:WAIT_RANDOM_DELAY
STATE:CREATE_PACKET
STATE:TRANSMIT
.
.
.
```

図 3.1: 従来のデバッグ方法

3.2 システムの構成

システム全体の構成を図 3.2 に示す。

提案システムは、先行研究システムに対してデバッグ支援機能を付加する形で構成されている。全体としては、通信プロトコルを実行する組込みシステムと、その動作を外部から確認するためのデバッグ支援システムからなる。

組込みシステム側では、状態遷移に基づいて通信プロトコルが実行され、その実行過程に関する情報がログとして出力される。一方、デバッグ支援システム側では、出力されたログを収集し、状態遷移の流れや実行状況を可視化する役割を担う。

3.3 各コンポーネントの概要

提案システムは、主に以下のコンポーネントから構成されている。

- 状態遷移図からコードを自動生成するコンポーネント

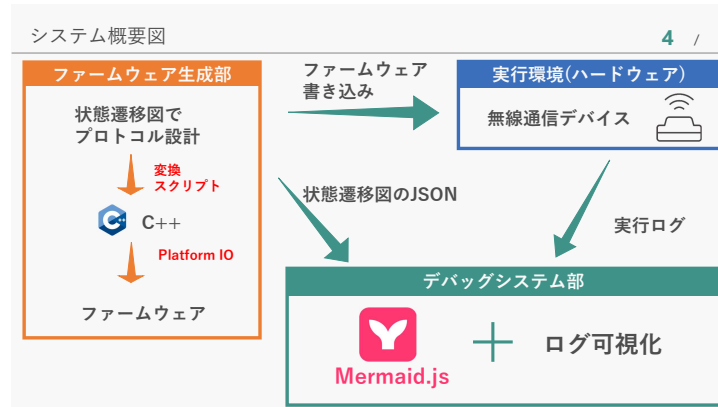


図 3.2: 提案システムの概要図

- 通信プロトコルを実行する組込みシステム
- デバッグ支援システム

3.3.1 コード生成コンポーネント

コード生成コンポーネントは、UMLの状態遷移図を解析し、通信プロトコルの動作を表現するプログラムコードを自動生成する役割を担う。本研究では、このコンポーネントとして共同研究（小林侑生ら）により開発された手法を利用している。本コンポーネントでは、状態遷移図の構造を解析し、存在する状態、遷移、イベントおよびアクションを抽出する。抽出した情報に基づき、通信プロトコルの動作を表現するC++コードを生成し、状態遷移はswitch-case文を用いて実装される。生成されたコードはPlatformIOを用いてビルドされ、無線通信デバイスに書き込まれた後、組込みシステム上で実行される。なお、本研究の主眼はコード生成手法そのものではなく、生成されたコードを用いた実機デバッグ支援環境の構築にあるため、コード生成手法の詳細については文献[?]を参照されたい。

3.3.2 組込みシステムコンポーネント

本システムにおける実行環境を図3.3に示す。本研究では、SAMD21マイコンと無線通信モジュールを一体化した無線通信デバイス（株式会社サーキットデザイン製どこでモ

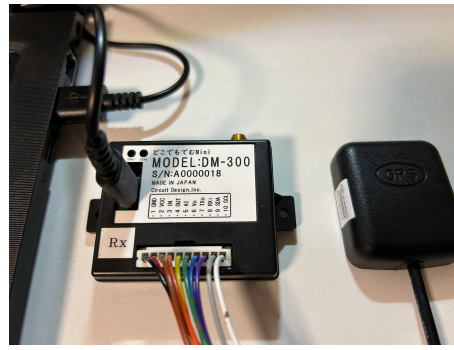


図 3.3: どこでモデム (左) と GPS モジュール (右)

デム) に GPS モジュールを組み合わせて、一つの無線通信デバイスとして使用した。この無線通信モジュールは 429MHz 帯で動作する汎用的なものであり、技術基準適合証明 (技適) を取得しているため、実際に電波を飛ばしながら通信プロトコルの検証を行うことができる。また、429MHz 帯は中山間地域において回折性能に優れており、低消費電力で長距離通信が可能な LPWA (Low Power Wide Area) 通信に適している。この特性を活かすことで、登山者見守りシステムなどへの応用も期待できる。さらに、本デバイスはマイコンと通信モジュールが一体化しているため、外部配線や追加ハードウェアを必要とせず、開発や実験環境の構築を容易に行うことが可能である。加えて、GPS モジュールを併用することで、位置情報の取得および 1PPS 信号による高精度な時刻同期を実現し、複数ノード間での時刻同期による無線通信プロトコルの実現を可能としている。GPS モジュールは使わない場合は外すこともできる。

3.3.3 デバッグ支援システムコンポーネント

最後に、本研究の中核であるデバッグ支援システムである。このコンポーネントは、組み込みシステムから出力されるログを収集し、状態遷移や内部動作を可視化する役割を担う。これにより、従来は把握が困難であったプロトコルの動作過程を確認できる。このコンポーネントは、次章で詳述するためここでは割愛する。

第 4 章

デバッグ支援システムの設計・実装

4.1 デバッグ支援システムの概要

本研究で提案するデバッグ支援システムは、ブラウザ上で動作するフロントエンドアプリケーションとして実装されている。本システムは、組込みシステムから得られる実行ログと状態遷移図情報を入力として、(1) 状態遷移の進行状況の可視化、(2) 変数値の追跡、(3) ステップ実行による逐次確認、を支援することを目的とする。

本章では、まずシステム全体の構成と動作モードを説明し、その後、各モードで利用する要素（ログ入力、ログ解析、変数追跡、状態遷移図描画、同期タイムライン生成）の設計と実装について述べる。

4.2 実装技術と採用理由

4.2.1 フロントエンドフレームワーク

UI およびロジックの構築には React.js（以下 React）を採用した。React はコンポーネント指向に基づくフロントエンドフレームワークであり、画面要素を状態と結び付けて管理できるという特徴を持つ。本研究で扱うデバッグ支援システムでは、状態遷移の進行に応じて表示内容が頻繁に変化するため、状態変化に伴う UI 更新を効率的に行える React は適している。また、状態遷移図のハイライト表示や変数値の動的更新などを宣言的に記述できる点も、本システムの可読性および保守性の向上に寄与している。

4.2.2 UIフレームワーク

UI のスタイリングおよびレイアウト設計には Bootstrap を使用した。Bootstrap は、あらかじめ定義された CSS クラスおよび UI コンポーネントを提供し、レスポンシブな画面設計を容易に実現できる。本システムでは、多数の情報（状態遷移図、ログ、変数値など）を同一画面上に整理して表示する必要があるため、グリッドシステムやカードコンポーネントを活用することで、視認性と操作性の両立を図った。

4.3 動作モードとシステム構成

4.3.1 動作モード

本システムは、単一ノードのログを対象とする**ログモード**と、2台のノードを同期させて解析する**同期モード**の2つの動作モードを備える。両モードは入力経路が異なる一方で、ログ解析に基づく変数追跡および状態遷移可視化は共通の処理として実装している。

4.3.2 モード間で共通に用いる要素

両モードで共通に用いる要素を以下に示す。

- ログの形式：状態遷移ログと変数更新ログ（4.6 節）
- ログ解析：状態列と変数値スナップショット列の構築（4.7 節）
- 変数追跡：ウォッチ機能による変数値表示（4.7 節内で説明）
- 状態遷移可視化：状態遷移図の描画とハイライト表示（??節）

一方、同期モードでは追加要素として、(1) ブラウザ上のシリアルモニタによる複数ポート受信、(2) タイムスタンプ付与と時系列統合による同期タイムライン生成、を行う（4.5 節）。

4.4 ログモードの設計・実装

4.4.1 ログモードの概要

ログモードは、単一ノードの動作を詳細に解析・可視化するための動作モードである。ユーザが実行ログファイルと状態遷移図 JSON ファイルをアップロードし、ログ解析によって得られた状態列と変数値スナップショット列を用いて、状態遷移図のハイライト表示および変数ウォッチ表示を行う。

4.4.2 ログモードのUI設計

ログモードのUIは、図4.1に示すように、主に以下の要素から構成されている。

- (1) 実行ログファイルをアップロードするためのファイルアップローダー
- (2) 状態遷移図の JSON ファイルをアップロードするためのファイルアップローダー
- (3) ステップ実行制御ボタン
- (4) スライダーによるステップジャンプコントロール
- (5) 変数のウォッチリスト表示エリア
- (6) 状態遷移ログ表示エリア
- (7) 状態遷移図表示エリア

ユーザは、(1)のファイルアップローダによって実行ログファイルを選択し、(2)のファイルアップローダによって状態遷移図 JSON ファイルを選択する。読み込まれた実行ログは解析され、状態遷移および変数スナップショットとして整理され、ログの再現に用いられる。状態遷移図の JSON ファイルは、デバッグシステム上に状態遷移図を描画するのに用いられる。(3)のステップ実行制御ボタンおよび(4)のスライダーを用いて、ユーザは状態遷移やその時点での変数値を逐次的に確認できる。(5)の変数ウォッチリストエリアには、現在のステップにおける変数名とその値が一覧表示される。(6)の状態遷移ログエリアには、解析された状態遷移ログが表示される。(7)の状態遷移図エリアには、状態遷移図が描画され、現在のステップに対応する状態がハイライト表示される。

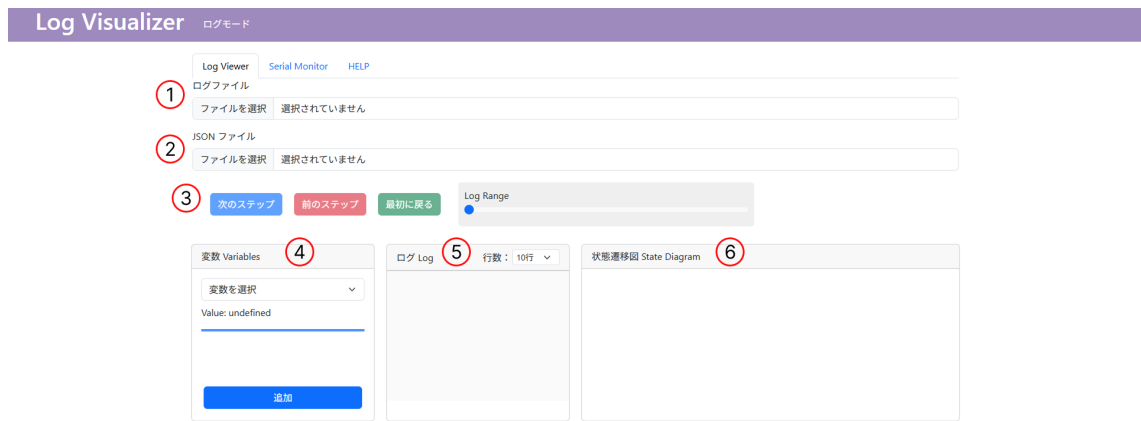


図 4.1: ログモードのレイアウト

4.5 同期モードの設計・実装

4.5.1 同期モードの概要

同期モードは、複数の無線通信デバイス間における挙動を比較・解析するための動作モードである。2台の無線通信デバイスをPCに接続し、それぞれのシリアルポートからログを同時に受信し、シリアルモニタで確認できる。受信ログはデバイスごとに管理し、ログ解析により状態と変数を抽出した後、受信時刻に基づいてイベント列を統合することで同期タイムラインを生成し、ある時点でそれぞれのデバイスがどういう状態にあるかを比較・解析できる。また、1台のデバイスに対しても、ログを受信し、状態遷移図の可視化デバッグを行うことができる。

4.5.2 同期モードのUI設計

同期モードでは、組み込みデバイスに接続して、ログを受信するシリアルモニタと、2台のデバイスの状態遷移を同期させて可視化デバッグするためのUIからなる。

シリアルモニタのUI

シリアルモニタのUIは図4.2に示すように、主に以下の要素から構成されている。

- ポート選択ドロップダウン

- 接続ボタンおよび切断ボタン
- シリアルモニタ表示エリア
- ログ解析ボタン（1 台用）
- 同期タイムラインを作成ボタン（2 台用）

ユーザは、組み込みデバイスを PC と USB 接続し、ポート選択ドロップダウンから接続するシリアルポートを選択する。接続ボタンを押下すると、ログの読み取りが開始され、切断ボタンを押下すると読み取りが中断される。各デバイスから送信されるログをリアルタイムに受信し、シリアルモニタエリアに逐次表示される。その後、1 台の場合は「ログ解析」ボタンを、2 台の場合は「同期タイムラインを作成」ボタンを押下することで、ログ解析および同期タイムライン生成が開始される。

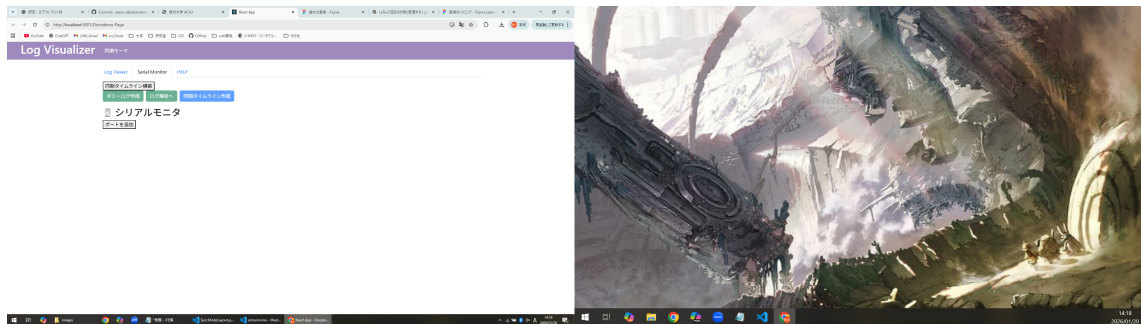


図 4.2: シリアルモニタのレイアウト

可視化デバッグの UI

可視化デバッグ UI は、図 4.3 に示すように、主に以下の要素から構成されている。

- ログモードと共通のステップボタンとスライダー
- 青枠内に示す、ノード 1 の状態遷移図 JSON アップローダ、変数ウォッチリスト、状態遷移図エリア
- 赤枠内に示す、ノード 1 とノード 2 の状態遷移が同期された状態遷移ログ
- 緑枠内に示す、ノード 2 の状態遷移図 JSON アップローダ、変数ウォッチリスト、状態遷移図エリア

ログモードと違い、同期モードでは先ほど示したシリアルモニタからログを受信し、ログ解析および同期タイムライン生成を行うため、ファイルアップローダはそれぞれのノードの状態遷移図 JSON ファイルのみとなっている。ノード1とノード2の状態遷移図はそれぞれ独立しており、各ノードの状態遷移図 JSON ファイルをアップロードすることで描画される。ステップボタンとスライダーはログモードと共通であり、同期タイムラインに基づいて両ノードの状態遷移および変数値を逐次的に確認できる。状態遷移ログエリアには、両ノードの状態遷移が受信時刻に基づいて統合された同期タイムラインが表示される。

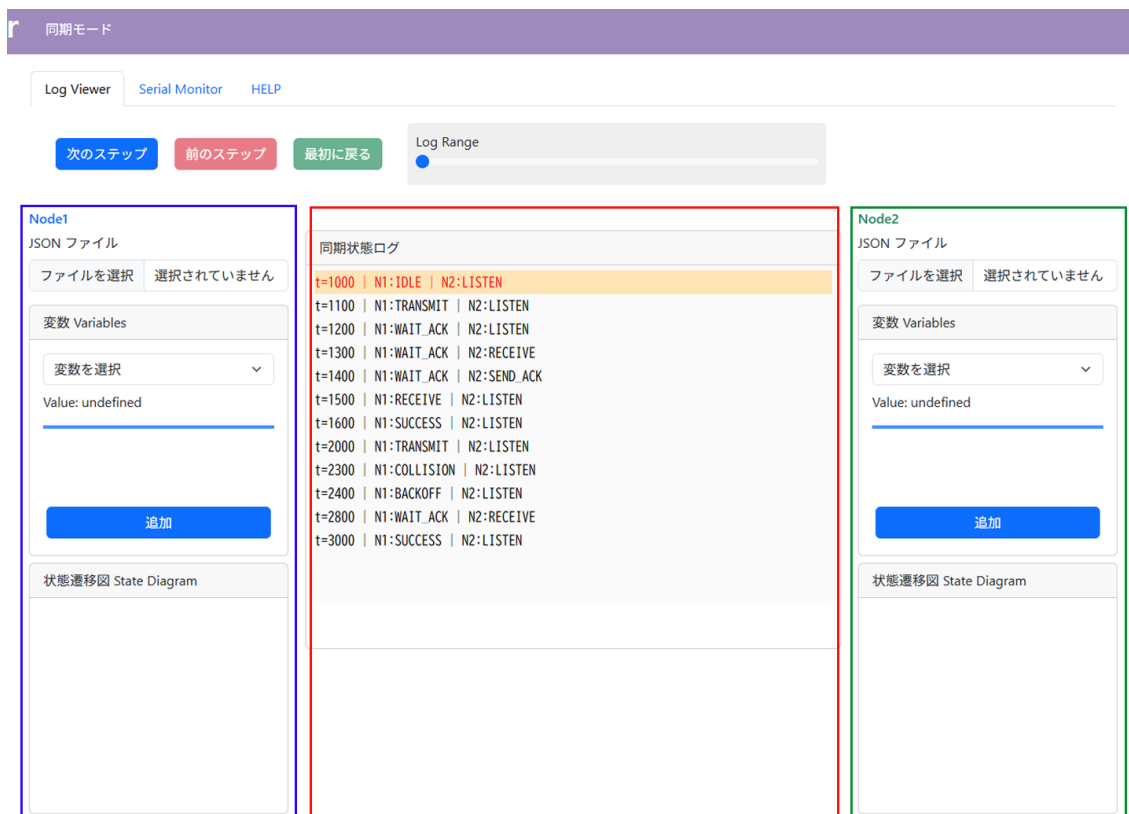


図 4.3: 同期モードのレイアウト

4.5.3 ブラウザ上でのシリアルモニタ実装

同期モードでは、専用のデスクトップアプリケーションを用いることなく、ブラウザ上でシリアル通信を扱えるように設計している。具体的には、Web Serial API を用いることで、Web ブラウザ上から直接シリアルポートに接続し、無線通信デバイスとの通信を実現した。ユーザはブラウザ上の UI から接続するシリアルポートを選択でき、open

ボタンおよび close ボタンを用いて接続および切断を制御する。接続中は各デバイスから送信されるログをリアルタイムに受信し、逐次表示できる。

4.5.4 同期タイムラインの生成

同期タイムライン生成処理では、各デバイスから受信したログ行に対し受信時刻を基準としたタイムスタンプを付与する。状態遷移ログおよび変数更新ログはイベントとして扱い、デバイスごとに時系列順に管理する。その後、複数デバイスのイベント列を時刻情報に基づいてマージし、全デバイスに共通の時系列イベント列を構築する。この統合イベント列から、各時刻におけるデバイスの状態スナップショットを再構成し、同期タイムラインとして可視化する。

図 4.4 および図 4.5 に、同期モードにおけるタイムライン表示例を示す。

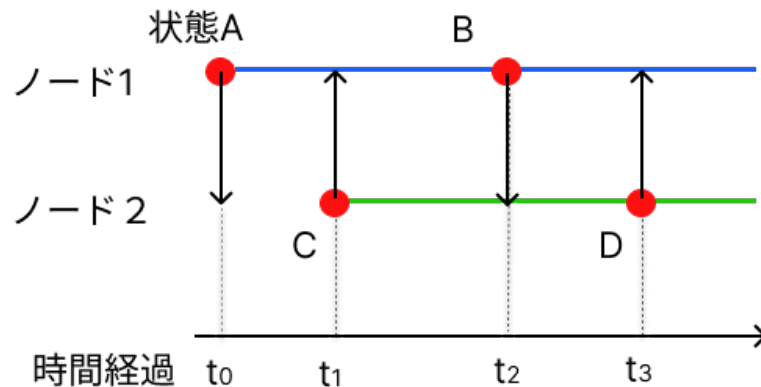


図 4.4: 同期モードのタイムライン

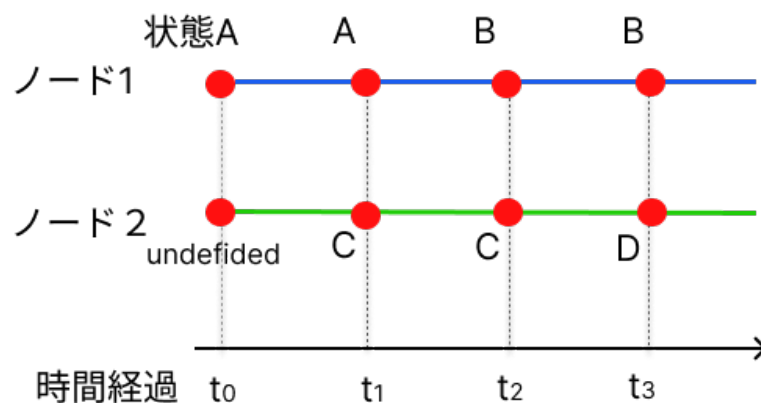


図 4.5: 同期モードのタイムライン

4.6 ログ設計

4.6.1 ログ生成の設計

ここでは、組込みシステムから生成されるログの設計について述べる。ログは、状態遷移イベント、変数変更イベント、およびその他ログから構成される。デバッグ支援システムは、状態遷移イベントと変数変更イベントのみを解析対象とし、その他ログは解析時に無視する。

状態遷移イベント

状態遷移イベントは、組込みシステムの実行中に状態が遷移した際に生成されるログであり、`STATE: 状態名` の形式で表現される。状態が切り替わるたびにファームウェア内の `updateState()` 関数が呼び出され、現在の状態をログ出力する設計とした。これにより、デバッグ支援システムは受信したログから現在の状態を把握し、状態遷移図上で該当する状態をハイライト表示できる。

変数変更イベント

変数変更イベントは、組込みシステム内の変数が変更された際に生成されるログであり、`[VAR:<name>=<value>]` の形式で表現される。変数の出力には C++ のマクロ機構を用いており、開発者は状態遷移図における各状態の `entry` 内に `PRINT_VAR_LOG(変数)` と記述するだけで、変数名と値を統一形式で出力できる。これによりデバッグ用コードの記述量を抑えつつ出力形式を統一でき、デバッグ支援システム側での解析と変数追跡を容易にする。

その他ログ

その他のログは、デバッグ情報やエラーメッセージなど、状態遷移イベントおよび変数変更イベント以外の情報を含む。これらは実行状況の把握やトラブルシューティングの補助として利用されるが、状態遷移や変数値の解析対象とはしておらず、解析の際には無視される。

ソースコード 4.1: 生成されるログの例

```
1 STATE: LISTEN
2 [VAR: var01=0]
3 [VAR: var02=10]
4 STATE: LISTEN
5 STATE: RECEIVE
6 [VAR: var01=1]
7 [VAR: var02=11]
8 STATE: TRANSMIT
9 [VAR: var01=2]
10 [VAR: var02=100]
```

4.7 ログ解析の設計（共通処理）

4.7.1 概要（解析の目的と処理の流れ）

本システムのログ解析は、ログ列を入力として状態遷移をステップ単位で再構成し、デバッグ表示に必要な内部情報（状態名と変数値）を整形する処理である。処理は「ログを1行ずつ読み取り、行種別（状態遷移／変数更新／その他）に応じて内部データを更新する」という流れで構成される。図4.6に、ログ解析全体の処理フローを示す。

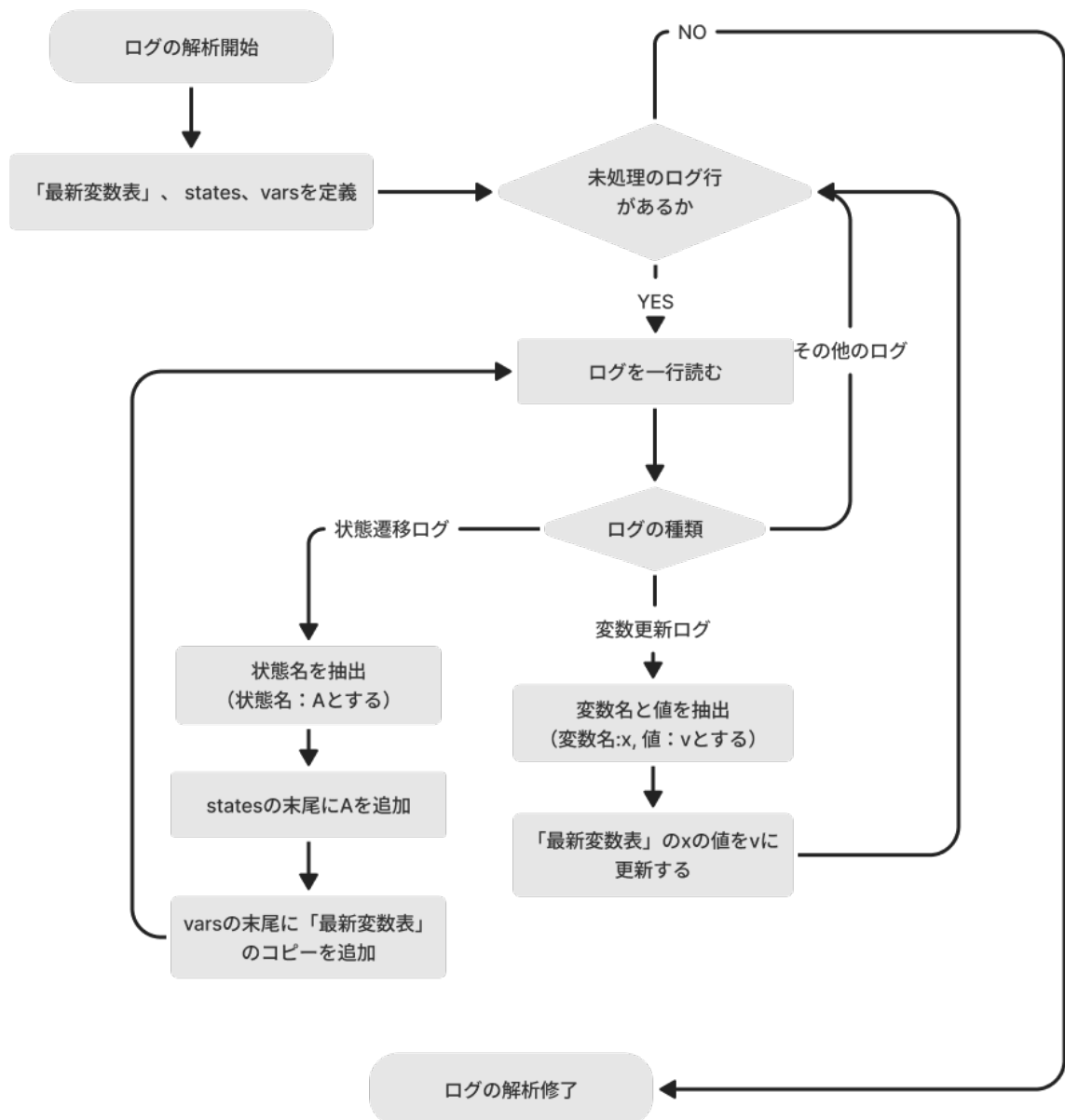


図 4.6: ログ解析の処理フロー

4.7.2 入力ログ形式とログ行の分類

ログは行単位のテキストとして扱い、各行は先頭文字列に基づき以下の3種類に分類される。

- 状態遷移ログ：STATE: で始まる行
- 変数更新ログ：[VAR: で始まる行
- その他のログ：上記のいずれにも該当しない行

状態遷移ログと変数更新ログのみを解析対象とし、デバッグメッセージやエラーメッセージ等のその他のログは無視する。

4.7.3 解析結果として構築するデータ構造

本システムはログを「状態遷移単位（ステップ）」で整理するため、解析結果として (i) 状態列 *states* と (ii) 変数値スナップショット列 *vars* を構築する。状態遷移ログの出現回数をステップ番号 *i* とみなし、*states[i]* に状態名を、*vars[i]* にその状態における変数値集合（連想配列）を格納する。

また、直近までに観測された変数値集合を保持する連想配列を「最新変数表」と呼ぶ（実装上は *lastVars* に相当する）。最新変数表は、後述する変数値スナップショットの再構成（継承）に用いる。

4.7.4 変数値スナップショットの再構成方針（継承と更新）

変数更新ログは「値が変更された変数のみ」を出力する設計であるため、各状態において全変数の値が毎回ログに現れるとは限らない。そこで本システムでは、各状態における変数値を次の規則で再構成する。

- 変数更新ログが出現した場合：最新変数表中の該当変数のみを更新する。
- 状態遷移ログが出現した場合：その時点の最新変数表をコピーし、当該ステップの変数値スナップショット *vars[i]* として確定する。

さらに、同一状態中に変数更新ログが続けて出現する場合があるため、直近のステップ *vars[i]* が既に生成されている状況では、最新変数表の更新内容を *vars[i]* にも反映する。これにより、「状態遷移直後に出力された変数更新」も、その状態の変数値として取り込める。図 4.7 に、変数値の継承（コピー）と更新の関係を示す。

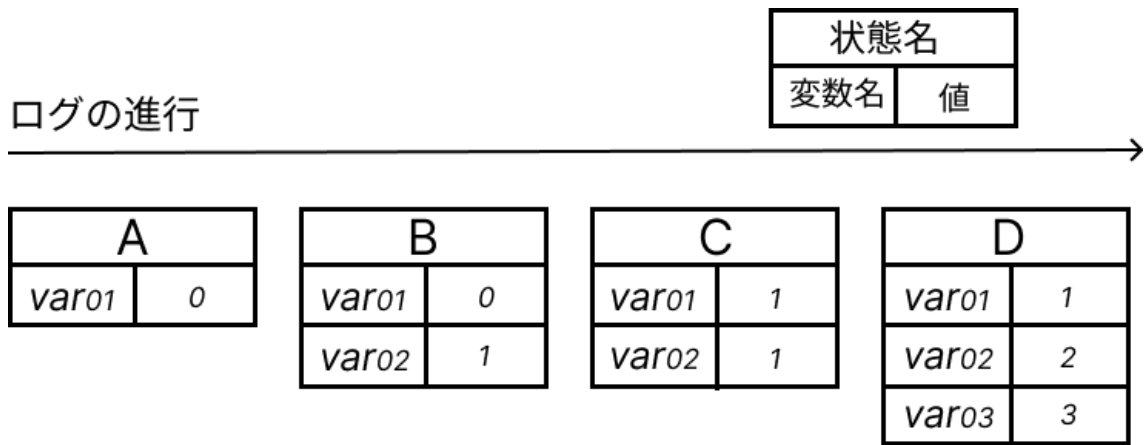


図 4.7: 変数値スナップショットの継承と更新

4.7.5 行処理アルゴリズム（擬似コード）

上記処理の擬似コードをソースコード 4.2 に示す。

ソースコード 4.2: ログ解析アルゴリズム（擬似コード）

```

1 states <- empty list
2 vars <- empty list
3 Vlatest <- empty map // 最新変数表（に相当）lastVars
4
5 for each line in log_lines:
6     if line is STATE:
7         s <- extract_state_name(line)
8         append(states, s)
9         append(vars, copy(Vlatest)) // スナップショット確定
10
11     else if line is VAR:
12         (x, v) <- extract_var_update(line)
13         Vlatest[x] <- v // 最新変数表を更新
14         if length(vars) > 0: // 直近ステップが存在する場合
15             vars[last_index][x] <- v // 直近スナップショットにも反映
16

```

```
17     else:
18         continue                // その他ログは無視
19
20 return (states, vars)
```

4.7.6 解析結果例

ログ解析によって得られる *states* と *vars* の例をソースコード 4.3 に示す。この例は、図 4.7 に示した「継承と更新」の関係に対応している。

ソースコード 4.3: ログ解析結果の例 (*states* と *vars*)

```
1 states = ["A", "B", "C", "D"]
2
3 vars = [
4     { var01: 0 },                // A のスナップショット
5     { var01: 0, var02: 1 },      // B のスナップショット
6     { var01: 1, var02: 1 },      // C のスナップショット
7     { var01: 1, var02: 2, var03: 3 } // D のスナップショット
8 ]
```

4.8 ステップ番号に基づく共通表示ロジック

4.8.1 共通の前提：ステップ番号 i と状態スナップショット

本システムでは、ログ解析により構築された状態列 *states*[i] と変数値スナップショット列 *vars*[i] (4.7 節) を基盤として、各種表示を統一的に制御する。ここで i は「状態遷移ログの出現順」に対応するステップ番号であり、ユーザがスライダー操作やボタン操作により i を変更すると、UI は *states*[i] および *vars*[i] を参照して表示を更新する。この設計により、ログモードと同期モードで入力経路が異なる場合でも、「ステップ番号 i を中心に表示を駆動する」という共通構造を維持できる。

4.8.2 ステップ実行のロジック

ステップ実行機能は、ステップ番号 i を1ずつ増減させることで、状態遷移の進行を逐次確認できるようにする機能である。ユーザ操作として、(1) 次のステップへ、(2) 前のステップへ、(3) 初期ステップへ、およびスライダーによる任意ステップへの移動を提供する。

内部的には、現在のステップ番号を i とし、 $0 \leq i \leq (N - 1)$ (N は状態列の長さ) を満たすよう境界条件を設ける。ユーザが操作を行うと i を更新し、それに伴い状態遷移ログ表示、変数ウォッチ表示、状態遷移図のハイライト表示を同時に更新する。

ソースコード 4.4: ステップ実行の擬似コード

```
1 # i: currentIndex, N: length(states)
2
3 onClickNext():
4     if i < N-1: i ← i + 1
5
6 onClickPrev():
7     if i > 0: i ← i - 1
8
9 onClickInitial():
10    i ← 0
11
12 onSliderChange(newI):
13    i ← clamp(newI, 0, N-1)
14
15 # i changes => update all views
16 render():
17     currentState ← states[i]
18     currentVars  ← vars[i]
```

4.8.3 変数ウォッチ機能のロジック

変数ウォッチ機能は、ユーザが監視したい変数（ウォッチリスト）を指定し、現在ステップ i における変数値 $vars[i]$ を参照して表示する機能である。ログは「変更された変数のみ」が出力されるが、ログ解析段階で変数値スナップショットを再構成しているため（4.7節）、表示側では単純に $vars[i]$ を参照するだけでよい。

ウォッチリストを $W = \{w_1, w_2, \dots\}$ とすると、各変数 w に対して $vars[i][w]$ を表示する。当該ステップで一度も観測されていない変数は未定義となり得るため、その場合は “—” などのプレースホルダ表示とする。

ソースコード 4.5: 変数ウォッチの擬似コード

```

1 #W: watch list (variable names)
2 # vars[i]: map of variable→value
3
4 renderWatch(i):
5   for each w in W:
6     if w in vars[i]:
7       show(w, vars[i][w])
8     else:
9       show(w, "—")

```

4.8.4 状態遷移表示機能（ハイライト表示）

状態遷移表示機能は、ステップ i に対応する現在状態 $states[i]$ を状態遷移図上でハイライトすることで、状態遷移の進行を視覚的に提示する機能である。ハイライトの色や強調表現の細部は割愛するが、基本的には「現在状態」と「直前状態」の関係に基づいて表示を変化させる。

具体的には、 $s_i = states[i]$ 、 $s_{i-1} = states[i-1]$ とし、 $s_{i-1} = s_i$ の場合は「同一状態の継続」とみなして強調表示を変更するなど、状態遷移の有無を表示上で区別する。

ソースコード 4.6: ハイライト判定の擬似コード


```
1 s <- states[i]
2 sp <- (i>0) ? states[i-1] : null
3
4 if sp == s:
5   highlight(s, "stay")  # 同一状態継続
6 else:
7   highlight(s, "move")  # 遷移先状態
```

4.8.5 Mermaid による状態遷移図描画（構文生成ロジック）

状態遷移図の描画には Mermaid.js を用いる。Mermaid は状態遷移図をテキスト（構文）として記述し、その文字列から図を生成するため、本システムでは、状態遷移図 JSON（状態集合・遷移集合・初期状態）から Mermaid 構文文字列を生成する。さらに、ステップ i に応じて現在状態をハイライトするスタイル指定を付加することで、ログと状態遷移図を連動させた表示を実現する。

構文生成は概ね次の手順で行う。(1) Mermaid のヘッダ（stateDiagram-v2 等）を出力、(2) 初期状態への遷移を出力、(3) 遷移集合を走査して矢印（from --> to）を列挙、(4) 現在状態 $states[i]$ に対するスタイル指定を付加、である。自己ループなど表示上不要な遷移は除外する。

ソースコード 4.7: Mermaid 構文生成の擬似コード

```
1 buildMermaid(stateJson, states, i):
2   M <- "stateDiagram-v2\n"
3   M <- M + "direction TB\n"
4   M <- M + "[*] --> " + stateJson.initialState + "\n"
5
6   for each (from,to) in stateJson.transitions:
7     if from != null and to != null and from != to:
8       M <- M + from + " --> " + to + "\n"
9
```

```
10   s <- states[i]
11   sp <- (i>0) ? states[i-1] : null
12
13   if sp == s:
14     M<- M+ "style " + s + " ... stay-style ...\n"
15   else:
16     M<- M+ "style " + s + " ... move-style ...\n"
17
18   return M
```

以上のように、本システムではステップ番号 i を中心に、(1) 状態列 $states[i]$ と (2) 変数スナップショット $vars[i]$ を参照して各表示（ログ表示・ウォッチ表示・状態遷移図表示）を一貫して更新する構造としている。

第 5 章

評価

5.1 評価の目的と方針

5.1.1 評価の目的（何を確かめたいか）

本章では、本研究で開発したデバッグ支援システムについて、提案機能が実際の実機ログ解析において有効に機能することを確認する。特に、(1) 状態遷移の追跡、(2) 変数値追跡（ウォッチ）、(3) ステップ実行に基づく逐次確認、および (4)（同期モードにおける）複数ノードの比較可能性、の観点から、実ログを用いて動作例を示し、定性的に評価する。

5.1.2 評価対象

評価対象は、ログ解析結果を用いてデバッグ表示を行う 2 つの動作モードである。ただし、本研究では「ログファイルを入力として行うデバッグ」と「単一ノードをシリアルモニタ経由で受信して行うデバッグ」は、ユーザが行う確認作業および画面上の操作がほぼ共通である。そこで本章では、単一ノードを対象とした評価を「単一ノードデバッグ」として共通化し、そのうえで同期モード固有の機能（同期タイムラインによる比較）を別途評価する構成とする。

- 単一ノードデバッグ（共通部分）
 - － 状態遷移の可視化（状態遷移図のハイライトとログ表示）

- 変数ウォッチ機能（状態ステップに対応した変数値追跡）
- ステップ実行および任意ステップへの移動（スライダ）
- 同期モード固有機能（差分）
 - 2 台のノードログを統合した同期タイムライン表示
 - 同一時刻における状態・変数の比較

5.1.3 評価で用いるデータ（実験対象）

評価には、実機で動作する無線通信プロトコルの実行ログと、対応する状態遷移図（JSON）を用いる。本研究ではログ設計として状態遷移ログ（STATE:）と変数更新ログ（[VAR:）を用いており、評価においても同形式のログを入力として用いる。また、同期モードの評価では2台のデバイスから得られたログを用いる。

5.2 単一ノードデバッグの評価（ログモード／単一ノード受信の共通部分）

5.2.1 評価シナリオ

本節では、単一ノードの実行ログを入力として、状態遷移の進行および変数値の推移をステップ単位で追跡できることを確認する。入力方法はログファイルアップロードであってもシリアルモニタ受信であっても、解析結果として構築される状態列 *states* と変数スナップショット列 *vars* を基に表示が行われるため、本節では両者に共通するデバッグ作業として記述する。

5.2.2 状態遷移の可視化（ハイライトとログ表示）

状態遷移ログを入力とし、状態遷移図上で現在状態がハイライト表示されること、および状態遷移ログ一覧がステップと同期して表示されることを確認する。また、連続して同一状態が出現する場合（自己遷移や待機状態の継続）においても、ステップの進行に応じて表示が更新されることを確認する。

5.2.3 変数ウォッチ機能（状態ステップに対応した変数値追跡）

変数更新ログを入力とし、各ステップにおいて変数値がスナップショットとして保持され、ウォッチ領域に表示されることを確認する。特に、変数更新ログが「変更された変数のみ」を出力する設計であるため、ログに現れない変数が直前の値を継承するという再構成方針により、状態ごとの変数値が一貫して表示されることを確認する。

5.2.4 ステップ実行（逐次確認）と任意ステップへの移動

次ステップ・前ステップの操作により、状態遷移図のハイライト、ログ表示、変数ウォッチ表示が同一のステップ番号（インデックス）に基づいて同期して更新されることを確認する。また、スライダ操作により任意ステップへ移動した場合も同様に、表示要素が同一インデックスに基づいて整合して更新されることを確認する。

5.3 同期モード固有機能の評価（2ノード同期タイムライン）

5.3.1 評価シナリオ

本節では、2台のデバイスから得られたログを受信し、時刻情報に基づき統合して同期タイムラインを生成できることを確認する。さらに、同一時刻における状態・変数の比較が可能となり、複数ノード間の相互作用やタイミング関係を把握しやすくなることを示す。

5.3.2 同期タイムライン表示

各デバイスの状態遷移・変数更新イベントを時系列に統合し、共通の時間軸上にスナップショットとして再構成できることを確認する。また、タイムライン上の各時刻を選択した際に、各ノードの状態・変数が対応付けられて表示されることを確認する。

5.3.3 複数ノード比較の有効性（定性的観察）

同期タイムラインにより、「同時刻における各ノードの状態と変数値」を同一画面で参

照できる。これにより、通信の送受信関係や待機状態の重なりなど、単一ノードでは判断しづらい時間関係を把握しやすくなることを確認する。

5.4 定性的評価

5.4.1 単一ノードデバッグに対する効果

- 状態遷移をステップ単位で追跡できることにより、実行の流れを視覚的に把握できる。
- 変数値を状態ごとのスナップショットとして追跡できることにより、ログ行を手作業で突き合わせる負担を軽減できる。
- ステップ実行により、状態遷移図・ログ・変数表示が同一インデックスで同期して更新され、逐次確認が容易になる。

5.4.2 同期デバッグに対する効果

- 複数ノードのログを統合して同期タイムラインとして可視化することで、同一時刻における状態・変数の比較が可能となる。
- 送受信や待機のタイミング関係を把握しやすくなり、分散システム特有の不具合解析を支援できる。

5.4.3 まとめ

以上より、本研究で提案するデバッグ支援システムは、単一ノードの逐次デバッグおよび複数ノードの同期比較の両面において、状態遷移と変数値を統合的に可視化し、デバッグ作業の把握性を向上させることを確認した。

第 6 章

考察

6.1 有効性の要因

6.1.1 状態遷移単位への再構成

本システムでは、ログを 1 行ずつ処理し、状態遷移ログの出現をステップ境界として扱うことで、実行を「状態遷移単位」に再構成した。これにより、ログ表示、状態遷移図ハイライト、変数ウォッチ表示を同一のステップ番号（インデックス）で一貫して同期できる構造となった。

6.1.2 変数値スナップショットの継承と更新

変数更新ログが「変更された変数のみ」を出力する設計であっても、最新変数表の保持とコピーにより、各ステップにおける変数値スナップショットを再構成できる。これにより、状態ごとの変数値を一貫して参照可能となり、ウォッチ機能として提示できた点が有効性の要因である。

6.1.3 同期モードにおける統合表示

同期モードでは、複数デバイスのログに時刻情報を付与し、統合イベント列を構築することで、同一時刻における状態・変数値の比較を可能とした。この「共通の時間軸に基づく比較」の導入により、分散システムの挙動把握が容易になった点が有効性の要因である。

6.2 システムの限界・制約

6.2.1 ログ設計への依存

本システムは、状態遷移ログ (STATE:) および変数更新ログ ([VAR:]) を前提として解析を行う。したがって、ログ形式が異なる場合には解析器の修正が必要であり、ログ設計への依存が制約となる。

6.2.2 同期タイムラインに関する制約

同期モードの統合は、ログ受信時刻に基づくタイムスタンプ付与により実現しているため、厳密な同時刻性を保証するものではない。この点は、同期表示の解釈における制約となる。

6.3 今後の課題

6.3.1 ログ形式・解析対象の拡張

今後は、ログ形式のバリエーション（例：複数種のイベント、構造化ログ）への対応や、解析対象の拡張（例：エラー原因推定を支援する追加情報）を検討する。

6.3.2 同期精度および比較支援の拡張

同期タイムラインにおける時間整合性の改善や、比較を支援する提示方法（差分強調やイベント関連付け）の拡張が課題である。

6.3.3 状態遷移図表現への対応

状態遷移図の表現力（例：複合状態や履歴状態）に対する対応範囲を整理し、必要に応じて描画・ハイライト手法を拡張することが課題である。

第 7 章

まとめ

7.1 本研究のまとめ

本研究では，無線センサネットワークの実機検証におけるデバッグ作業を支援するため，ブラウザ上で動作するデバッグ支援システムを設計・実装した。本システムは，状態遷移ログおよび変数更新ログを解析し，状態遷移の可視化，変数値追跡，ステップ実行を提供する。さらに，同期モードにより複数ノードのログを統合し，同一時間軸上での比較を可能とした。

7.2 得られた知見と今後の展望

評価を通じて，単一ノードの逐次デバッグおよび複数ノードの同期比較の双方において，状態遷移と変数値を統合的に可視化できることを確認した。今後は，ログ形式・解析対象の拡張，同期精度の改善，および状態遷移図表現への対応範囲の拡張を進める。

参考文献

- [1] IoT Analytics: "State of IoT 2024: Number of Connected IoT Devices Grows 16% to 16.6 Billion", 2024.
- [2] Grand View Research: "Industrial Wireless Sensor Network Market Size, Share & Trends Analysis Report, 2024–2030", 2024.
- [3] 旭 健汰, アサノ デービッド, 不破 泰: "無線モデムを用いたセンサネットワーク MAC プロトコル検証システムの開発", 信学技報, NS2023-147, pp.120–125, Dec. 2023.
- [4] 小林 遼, アサノ デービッド, 不破 泰: "センサーネットワーク検証システムにおける遷移図を用いた MAC プロトコル設計環境の開発", 信学技報, NS2023-148, pp.126–131, Dec. 2023.
- [5] 小林 侑生, アサノ デービッド, 不破 泰:
- [6] PlatformIO Labs, "What is PlatformIO?", <https://docs.platformio.org/en/latest/what-is-platformio.html>, 参照 Oct. 15, 2025.
- [7] Mermaid.js, "Overview," <https://mermaid.js.org/intro/>, 参照 Oct. 15, 2025.
- [8] Mermaid.js, "State Diagram Syntax (stateDiagram-v2)," <https://mermaid.js.org/syntax/stateDiagram.html>, 参照 Oct. 15, 2025.
- [9] Eclipse Foundation, "MQTT: Message Queuing Telemetry Transport," <https://mqtt.org/>, 参照 Oct. 15, 2025.
- [10] 園田 継一郎, 不破 泰, アサノ デービッド, "無線センサーネットワークの端末・中継機における送信タイミング決定時間短縮方法の検討," 信学技報, NS2022-138, Dec. 2022.